

Full Team Task Breakdown

✅ Person 1 — DONE

- Next.js project setup
 - Prisma schema (User, Post, Comment, Like, Follow)
 - Database migration
 - seed.js with fake data
-

👤 Person 2 — Auth + User APIs

Files to create:

- repo/userRepository.js — functions: getUserById, getUserByEmail, createUser, updateUser, toggleFollow, getAllUsers
 - app/api/auth/login/route.js — POST: receive email + password, find user by email, check password matches, return user object
 - app/api/auth/register/route.js — POST: receive username/email/password, check no duplicates, create user, return user
 - app/api/users/[id]/route.js — GET: return user profile with follower/following/post counts
 - app/api/users/[id]/profile/route.js — PUT: update username, email, bio
 - app/api/users/[id]/follow/route.js — POST: toggle follow/unfollow, return { following: true/false }
 - app/api/users/route.js — GET: return all users (for suggested users list)
-

👤 Person 3 — Posts + Comments + Likes APIs

Files to create:

- repo/postRepository.js — functions: getAllPosts, getPostById, getPostsByUser, createPost, deletePost, toggleLike, isLikedByUser
- repo/commentRepository.js — functions: getCommentsByPost, createComment

- `app/api/posts/route.js` — GET: return all posts with author info + like/comment counts. POST: create a new post
 - `app/api/posts/[id]/route.js` — GET: return single post with details. DELETE: delete a post
 - `app/api/posts/[id]/like/route.js` — POST: toggle like, return { liked: true/false, likeCount: n }
 - `app/api/posts/[id]/comments/route.js` — GET: return all comments for a post. POST: add a comment
-

Person 4 — Statistics Page

Files to create:

- `repo/statsRepository.js` — 6 query functions (all must use Prisma queries, no manual filtering in JS):
 1. Average number of followers per user
 2. Average number of posts per user
 3. Most active user in the last 3 months (most posts)
 4. Most liked post
 5. Most commented post
 6. Top 5 users with the most followers
- `app/api/stats/route.js` — GET: call all 6 functions and return results as JSON
- `app/stats/page.jsx` — React page that fetches from `/api/stats` and displays each stat in a card